

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA: Raffi Aditiya Saputra

NIM: 109082500094

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dasar teori yang dijelaskan dalam modul ini berfokus pada algoritma pengurutan (sorting), khususnya *Selection sort* dan *Insertion Sort*.

Selection Sort : ide utamanya adalah mencari nilai ekstrem (misalnya nilai terkecil untuk ascending) dari kumpulan data, lalu menempatkannya di posisi yang seharusnya. Proses ini dilakukan berulang kali hingga seluruh data terurut.

Insertion Sort : Berbeda dengan Selection Sort, insertion sort bekerja dengan cara menyisipkan elemen yang sudah terurut. Caranya adalah dengan menggeser elemen-elemen yang lebih besar (atau lebih kecil, tergantung ascending/descending) ke kanan, sehingga tersedia ruang kosong untuk menepatkan elemen baru. Algoritma ini melibatkan pencarian posisi dengan sequential search dan proses penyisipan.

B. Guided

1. [Program selection sort]

```
package main

import "fmt"

func selecectionSortArray(angka *[5]int) {
    var idx_min, i, j int
    for i = 0; i < len(angka)-1; i++ { // Loop Luar (iterasi sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { // Loop dalam (mencari nilai ekstrim)
            if angka[j] <= angka[idx_min] { // sorting terkecil ke terbesar
                idx_min = j
            }
        }
    }
}
```

```

        }

    }

    if idx_min != i {
        angka[i], angka[idx_min] = angka[idx_min], angka[i]
    }
}

}

func main() {
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukan data angka ke-%d : ", i)

        fmt.Scan(&arrAngka[i])
    }

    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")

    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORTING ===")

    selectionSortArray(&arrAngka)

    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output

```

PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
\soal1.go"
Masukan data angka ke-0 : 5
Masukan data angka ke-1 : 3
Masukan data angka ke-2 : 4
Masukan data angka ke-3 : 2
Masukan data angka ke-4 : 1

=== SEBELUM DISORTING ===
5 - 3 - 4 - 2 - 1 -
=== SETELAH DISORTING ===
1 - 2 - 3 - 4 - 5 -
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>

```

Penjelasan:

Program tersebut merupakan implementasi algoritma Selection Sort untuk mengurutkan array berisi lima bilangan bulat. Prosesnya dimulai dengan meminta pengguna memasukkan lima angka, lalu angka-angka tersebut ditampilkan sebelum dilakukan pengurutan. Fungsi `selectionSortArray` bekerja dengan cara mencari nilai terkecil dari sisa elemen array pada setiap iterasi, kemudian menukar posisi elemen terkecil tersebut dengan elemen pada indeks saat ini. Dengan pengulangan hingga akhir array, hasil akhirnya adalah data yang terurut dari nilai terkecil ke terbesar

2. [Program selection sort]

```

package main

import "fmt"

const NMAX = 1000000

type partai struct {
    nama int
    suara int
}

type tabPartai [NMAX]partai

func posisi(t tabPartai, n int, nama int) int {

    for i := 0; i < n; i++ {

        if t[i].nama == nama {

```

```

        return i
    }

}

return -1
}

func main() {
    var p tabPartai

    var x, n, idx int

    fmt.Println("Masukan proses input suara:")

    n = 0

    fmt.Scan(&x)

    for x != -1 && n < NMAX {

        idx = posisi(p, n, x)

        if idx != -1 {

            p[idx].suara++

        } else {

            p[n].nama = x

            p[n].suara = 1

            n++

        }

        fmt.Scan(&x)

    }

    var temp partai

    for i := 1; i < n; i++ {

        temp = p[i]

        j := i - 1

        for j >= 0 && p[j].suara < temp.suara {

            p[j+1] = p[j]

            j--

        }

        p[j+1] = temp
    }
}

```

```

    }

    fmt.Println("Hasil Perhitungan suara:")

    for i := 0; i < n; i++ {

        if i > 0 {

            fmt.Print(" ")

        }

        fmt.Printf("%d(%d)", p[i].nama, p[i].suara)

    }

    fmt.Println()
}

```

Output :

```

PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
\soal2.go"
5
7 2 9 4 1
9 7 4 2 1
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
\soal2.go"
6
10 3 15 8 6 20
20 15 10 8 6 3
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>

```

Penjelasan:

Program tersebut adalah implementasi algoritma Insertion Sort yang digunakan untuk mengurutkan sekumpulan bilangan bulat secara descending atau dari nilai terbesar ke nilai terkecil. Cara kerjanya, setiap elemen array akan dibandingkan dengan elemen sebelumnya, lalu digeser ke kanan jika elemen yang sedang diproses lebih besar. Setelah posisi yang tepat ditemukan, elemen tersebut dimasukkan sehingga urutan tetap terjaga. Pada fungsi main, pengguna diminta memasukkan jumlah data dan nilai-nilai bilangan bulat, kemudian data tersebut diproses oleh fungsi insertionSort. Hasil akhirnya ditampilkan berupa deretan angka yang sudah terurut dari besar ke kecil.

3. [judul guided 3]

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {

    var temp string

    var i, j int

    i = 1

    for i <= n-1 {

        j = i

        temp = T[j]

        for j > 0 && temp < T[j-1] {

            T[j] = T[j-1]

            j = j - 1

        }

        T[j] = temp

        i = i + 1

    }

}

func main() {

    var data arrString

    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        fmt.Scan(&data[i])

    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {

        fmt.Print(data[i], " ")

    }

}
```

Output :

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run '
unnerFile.go'
5
apel jeruk mangga pisang anggur
anggur apel jeruk mangga pisang
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run '
unnerFile.go'
4
ayam kuda sapi zebra
ayam kuda sapi zebra
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>
```

Penjelasan: Program ini berfungsi mengurutkan kumpulan kata berdasarkan urutan abjad dari yang paling awal sampai paling akhir menggunakan metode / *insertion Sort*. Pertama program membaca jumlah kata yang akan dimasukkan, lalu menyimpan semua kata tersebut ke dalam wadah yang sudah disiapkan dengan kapasitas maksimal 100 data.

2. Program Selection Sort Struct

Jawaban :

```
package main
import "fmt"
type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { // Loop luar:
menentukan posisi yang akan diisi
        idx_min = i // Asumsikan elemen pertama adalah
nilai terkecil

        // Loop dalam: mencari indeks dengan nilai
terkecil
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK { //
Membandingkan IPK
                idx_min = j // Simpan indeks nilai
terkecil yang baru
            }
        }
    }
}
```



```

        // Tukar posisi jika ditemukan nilai yang lebih
        kecil
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min],
sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    // Menginput data mahasiswa
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d -
--\n", i+1) // Diubah ke i+1 agar nomor mulai dari 1
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }

    // Menampilkan data sebelum diurutkan
    fmt.Println()
    fmt.Println(" === SEBELUM SORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n",
i+1)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    // Memanggil fungsi pengurutan
    SelectionSortArray(&structMHS)

    // Menampilkan data setelah diurutkan
    fmt.Println(" === SETELAH SORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n",
i+1)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")

```

```
        fmt.Println()  
    }
```

Output :

```

--- Masukkan Data Mahasiswa Ke-1 ---
Nama : Raffi
NIM : 109082500094
IPK : 3.6
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : Hanif
NIM : 109082500058
IPK : 3.5
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : Sigit
NIM : 123
IPK : 3.4
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : Asoy
NIM : 3.3
IPK : 3.3
--- Masukkan Data Mahasiswa Ke-5 ---
Nama : Melehoy
NIM : 123
IPK : 3.2

=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-1 ---
Nama : Raffi
NIM : 109082500094
IPK : 3.60
--- Data Mahasiswa Ke-2 ---
Nama : Hanif
NIM : 109082500058
IPK : 3.50
--- Data Mahasiswa Ke-3 ---
Nama : Sigit
NIM : 123
IPK : 3.40
--- Data Mahasiswa Ke-4 ---
Nama : Asoy
NIM : 3.3
IPK : 3.30
--- Data Mahasiswa Ke-5 ---
Nama : Melehoy
NIM : 123
IPK : 3.20
=====

```

Penjelasan :

Program ini dimulai dengan mendeklarasikan paket utama dan memanggil pustaka fmt yang berfungsi untuk menangani proses input dan output data. Selanjutnya dibuatkan struktur data bernama mahasiswa yang berisi tiga

variabel yaitu nama dan nim bertipe teks, serta IPK bertipe angka desimal. Struktur ini berfungsi sebagai cetakan atau kerangka untuk menyimpan data mahasiswa secara lengkap dalam satu wadah.

1. Unguided

Jawaban :

```
package main

import "fmt"

// Fungsi selection sort untuk mengurutkan nomor rumah
membesar

func selectionSort(rumah []int) {
    var idxTerkecil, i, j int
    // Perulangan untuk menentukan posisi yang akan diisi
    for i = 0; i < len(rumah)-1; i++ {
        idxTerkecil = i

        for j = i + 1; j < len(rumah); j++ {
            if rumah[j] < rumah[idxTerkecil] {
                idxTerkecil = j // Simpan indeks data yang
lebih kecil
            }
        }

        // Tukar posisi data terkecil ke posisi yang
seharusnya
        if idxTerkecil != i {
            rumah[i], rumah[idxTerkecil] =
rumah[idxTerkecil], rumah[i]
        }
    }
}

func main() {
```

```

    var n int
    // Membaca jumlah daerah
    fmt.Scan(&n)

    // Memproses setiap daerah
    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m) // Membaca jumlah rumah di daerah
        tersebut

        // Membuat wadah untuk menampung nomor rumah
        daftarRumah := make([]int, m)

        // Membaca semua nomor rumah
        for j := 0; j < m; j++ {
            fmt.Scan(&daftarRumah[j])
        }

        // Mengurutkan nomor rumah menggunakan selection
        sort
        selectionSort(daftarRumah)

        // Menampilkan hasil urutan
        for k := 0; k < m; k++ {
            if k > 0 {
                fmt.Print(" ") // Beri spasi antar nomor
            }
            fmt.Print(daftarRumah[k])
        }
        fmt.Println()
    }
}

```

Output :

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal1.go"
3
5 2 1 7 9 13
1 2 7 9 13
15 27 39 75 133 189
1 4 9
1 2 7 9 13
6 189 15 27 39 75 133
1 1 2 4 6 7 9 9 13 27 39 75 133 189 189
```

Penjelasan :

Program ini diawali dengan mendeklarasikan paket utama dan memuat pustaka `fmt` untuk keperluan proses baca dan tulis data. Selanjutnya dibuatkan fungsi bernama `selection sort` yang menerima sebuah kumpulan bilangan berupa nomor rumah. Di dalam fungsi ini terdapat variabel penanda indeks nilai terkecil serta variabel pengendali perulangan. Proses pengurutan dimulai dari data paling awal, di mana pada setiap langkahnya nilai pada posisi tersebut langsung dianggap sebagai nilai yang paling kecil.

2. Soal Unguided 3

[isi dengan soal unguided 3]

Jawaban :

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        if idxMin != i {
            arr[i], arr[idxMin] = arr[idxMin], arr[i]
        }
    }
}
```

```

    }
}

func selectionSortDesc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        if idxMax != i {
            arr[i], arr[idxMax] = arr[idxMax], arr[i]
        }
    }
}

func main() {
    reader := bufio.NewReader(os.Stdin)
    writer := bufio.NewWriter(os.Stdout)
    defer writer.Flush()

    var n int
    _, err := fmt.Fscan(reader, &n)
    if err != nil || n <= 0 || n >= 1000 {
        return
    }

    for i := 0; i < n; i++ {
        var m int
        fmt.Fscan(reader, &m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nilai int
            fmt.Fscan(reader, &nilai)
            if nilai%2 == 0 {
                genap = append(genap, nilai)
            } else {
                ganjil = append(ganjil, nilai)
            }
        }

        selectionSortAsc(ganjil)
        selectionSortDesc(genap)

        isFirst := true
    }
}

```

```

        for _, nilai := range ganjil {
            if !isFirst {
                fmt.Fprint(writer, " ")
            }
            fmt.Fprint(writer, nilai)
            isFirst = false
        }
        for _, nilai := range genap {
            if !isFirst {
                fmt.Fprint(writer, " ")
            }
            fmt.Fprint(writer, nilai)
            isFirst = false
        }
        fmt.Fprintln(writer)
    }
}

```

Output :

```

PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal2.go
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 7 9 13 2
15 27 39 75 133 189
1 9 4
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>

```

Penjelasan :

Program dimulai dengan mendeklarasikan paket utama dan mengimpor tiga pustaka pendukung yaitu bufio, fmt, dan os. Pustaka ini digunakan agar proses pembacaan masukan dan penulisan keluaran menjadi jauh lebih cepat dan efisien, yang sangat berguna ketika jumlah data yang diproses berukuran besar seperti yang disyaratkan dalam soal.

3. Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban:

```
package main

import (
    "fmt"
)

func main() {
    var input int
    var data []int

    for {
        _, err := fmt.Scan(&input)
        if err != nil {
            break
        }
        if input < 0 {
            break
        }
        data = append(data, input)
    }

    if len(data) == 0 {
```

```
        return

    }

    insertionSort(data)

    for i, val := range data {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(val)
    }
    fmt.Println()

    statusjarak(data)
}

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j = j - 1
        }
    }
}
```

```
    arr[j+1] = key
}
}

func statusjarak(arr []int) {
    n := len(arr)
    if n <= 1 {
        fmt.Println("Data berjarak 0")
        return
    }

    jarakAwal := arr[1] - arr[0]
    isiTetap := true

    for i := 1; i < n-1; i++ {
        if arr[i+1]-arr[i] != jarakAwal {
            isiTetap = false
            break
        }
    }

    if isiTetap {
        fmt.Printf("Data berjarak %d\n", jarakAwal)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}
```

```
}  
}
```

Output:

```
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run  
soal3.go"  
31 13 25 43 1 7 19 37 -5  
1 7 13 19 25 31 37 43  
Data berjarak 6  
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> |
```

Penjelasan: Program ini dimulai dengan mendeklarasikan paket utama dan memuat pustaka fmt untuk keperluan membaca masukan dan menampilkan keluaran. Di dalam fungsi utama, dideklarasikan variabel input untuk menampung angka yang diketik pengguna, serta variabel data berupa kumpulan bilangan yang berfungsi sebagai wadah penyimpanan seluruh angka yang masuk.

4. Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919  
type Buku = <  
  id, judul, penulis, penerbit : string  
  eksemplar, tahun, rating : integer >  
type DaftarBuku = array [ 1..nMax] of Buku  
Pustaka : DaftarBuku  
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir

Jawaban:

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating    int
}

type DaftarBuku [nMax]Buku

func main() {
    var pustaka DaftarBuku
    var nPustaka int
    var ratingDicari int
```

```

reader := bufio.NewReader(os.Stdin)

fmt.Scanln(&nPustaka)

BacaDaftarBuku(&pustaka, nPustaka, reader)

inputRating, _ := reader.ReadString('\n')
inputRating = strings.TrimSpace(inputRating)
ratingDicari, _ = strconv.Atoi(inputRating)

CetakTerfavorit(pustaka, nPustaka)
UrutBuku(&pustaka, nPustaka)
Cetak5Terbaru(pustaka, nPustaka)
CariBuku(pustaka, nPustaka, ratingDicari)
}

func BacaDaftarBuku(pustaka *DaftarBuku, n int, reader *bufio.Reader) {
    for i := 0; i < n; i++ {
        var b Buku

        b.id, _ = reader.ReadString('\n')
        b.id = strings.TrimSpace(b.id)

        b.judul, _ = reader.ReadString('\n')
        b.judul = strings.TrimSpace(b.judul)

        b.penulis, _ = reader.ReadString('\n')

```

```

        b.penulis = strings.TrimSpace(b.penulis)

        b.penerbit, _ = reader.ReadString('\n')
        b.penerbit = strings.TrimSpace(b.penerbit)

        txtEksemplar, _ := reader.ReadString('\n')
        b.eksemplar, _ = strconv.Atoi(strings.TrimSpace(txtEksemplar))

        txtTahun, _ := reader.ReadString('\n')
        b.tahun, _ = strconv.Atoi(strings.TrimSpace(txtTahun))

        txtRating, _ := reader.ReadString('\n')
        b.rating, _ = strconv.Atoi(strings.TrimSpace(txtRating))

        pustaka[i] = b
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
    maxIdx := 0
    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {

```

```

        maxIdx = i
    }
}

fav := pustaka[maxIdx]

fmt.Printf("%s, %s, %s, %d\n", fav.judul, fav.penulis, fav.penerbit, fav.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }
}

```



```
var judulList []string

for i := 0; i < limit; i++ {

    judulList = append(judulList, pustaka[i].judul)

}

fmt.Println(strings.Join(judulList, " "))
}
```

```
func CariBuku(pustaka DaftarBuku, n int, r int) {

    left := 0

    right := n - 1

    foundIdx := -1

    for left <= right {

        mid := left + (right-left)/2

        if pustaka[mid].rating == r {

            foundIdx = mid

            break

        } else if pustaka[mid].rating < r {

            right = mid - 1

        } else {

            left = mid + 1

        }

    }

    if foundIdx != -1 {
```

```

    b := pustaka[foundIdx]

    fmt.Printf("%s, %s, %s, %d, %d, %d\n", b.judul, b.penulis, b.penerbit, b.tahun,
b.eksemplar, b.rating)

} else {

    fmt.Println("Tidak ada buku dengan rating seperti itu")

}

}

```

Output:

```

PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA> go run
soal4.go
3
B01
Laskar Pelangi
Andrea Hirata
Bentang Pusaka
10
2006
92
B02
Bumi Manusia
Pramoedya Ananta Toer
Lentera Dipantara
5
1980
95
B03
Filosofi Kopi
Dee Lestari
Truedee Books
15
2006
88
92
Bumi Manusia, Pramoedya Ananta Toer, Lentera Dipantara, 1980
Bumi Manusia Laskar Pelangi Filosofi Kopi
Laskar Pelangi, Andrea Hirata, Bentang Pusaka, 2006, 10, 92
PS C:\Users\MUHASYIM\Documents\109082500094_RAFFI-ADITIYA-SAPUTRA>

```

Penjelasan:

Program tersebut adalah sistem sederhana untuk mengelola data buku dalam sebuah pustaka. Pertama program mendefinisikan tipe data Buku yang berisi informasi seperti id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Semua buku disimpan dalam array bernama DaftarBuku. Pada fungsi main, pengguna diminta memasukkan jumlah buku yang akan dicatat (nPustaka)., kemudian setiap detail buku dibaca satu per satu melalui fungsi BacaDaftarBuku. Setelah semua data masuk, program juga membaca sebuah nilai rating yang akan digunakan untuk pencarian.

Kesimpulan:

Kesimpulan dari praktikum pada modul tersebut bahwa algoritma sorting seperti selection sort dan insertion sort digunakan untuk mengurutkan data dengan cara yang berbeda, namun sama-sama bertujuan menghasilkan data yang terstruktur rapi. Selection sort bekerja dengan cara yang berbeda dengan mencari nilai ekstrem (terkecil atau terbesar) lalu menukarnya ke posisi yang sesuai, sedangkan insertion sort menyisipkan elemen ke posisi yang tepat di antara data yang sudah terurut.